SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES COMPUTER SCIENCE AND ENGINEERING

CSA47 – DEEP LEARNING

ASSIGNMENT – 1

CO1 – MATHEMATICAL FOUNDATIONS AND MACHINE LEARNING PRINCIPLES

Nature: Individual **Name:** Pola Nandivardhan **Reg. No:** 192324270

Duration: 1 Week Total Marks: 50

CO1: Analyze the foundational mathematical concepts, including linear algebra and probability, and core machine learning principles like bias-variance trade-offs and supervised/unsupervised learning algorithms.

Topics: Linear Algebra, Probability and Information Theory, Numerical Computation, Capacity, Overfitting/Underfitting, and Bayesian Statistics.

Purpose

Investigate how mathematical foundations influence the reliability of learning systems. Students must connect equations, assumptions and numerical behaviour to practical model decisions.

Scenario: Healthcare Risk Prediction

A hospital plans to develop an intelligent system to predict disease risk using patient age, blood pressure, glucose level, cholesterol and family history. The dataset contains labelled patient records as well as unlabelled records. You are required to analyze the data, select suitable learning approaches and justify decisions using mathematical reasoning.

TASK 1 – Patient Data Representation and Normalization | 5 Marks

A hospital records age, blood pressure, glucose level and cholesterol for three patients. These features have different numerical scales.

a) Explain the likely effect of using the raw features without centering or normalization.

b) Represent the patient data as a feature matrix and apply a suitable normalization procedure using a small numerical example.

c) Explain why normalization is important for distance-based and gradient-based learning methods.

Evidence Expected: Feature matrix, normalization procedure, numerical verification, explanation of scale differences and interpretation.

Answer – Task 1

a) Effect of using raw features without centering or normalization

The four features are measured in different units and ranges (age in years, blood pressure in mmHg, glucose and cholesterol in mg/dL). If they are used directly:

- Features with large numerical values (glucose, cholesterol) dominate distance calculations, while features with small values (age, blood pressure) are almost ignored, even if they are medically important.
- The result depends on the units chosen (for example mg/dL against mmol/L), which has no medical meaning.
- Gradient-based learning becomes slow and unstable because the loss surface is stretched in the directions of the large-scale features, so the weights zig-zag towards the minimum.
- Values that are not centered around zero can saturate activation functions such as sigmoid and tanh.

Example: for patients 1 and 2 below, the squared raw difference is 52.9% due to glucose and 33.9% due to cholesterol, but only 4.8% due to age and 8.5% due to blood pressure.

b) Feature matrix and normalization

Let X be the feature matrix with one row per patient and one column per feature (3 × 4):

Patient	Age (years)	Blood pressure (mmHg)	Glucose (mg/dL)	Cholesterol (mg/dL)
P1	45	130	110	220
P2	60	150	160	260

P3	30	120	90	180

Procedure: z-score normalization (centering and scaling) is applied to each column j:

$z = (x - \mu j) / \sigma j$, where μj = column mean and σj = column standard deviation

Worked example for the Age column: $\mu = (45 + 60 + 30)/3 = 45$ and $\sigma = \sqrt{[(0^2 + 15^2 + (-15)^2)/3]} = \sqrt{150} = 12.247$. Therefore z for age 60 = $(60 - 45)/12.247 = 1.225$, for age 30 it is -1.225 and for age 45 it is 0. The other columns are calculated in the same way.

Column means: Age = 45, BP = 133.33, Glucose = 120, Cholesterol = 220. Column standard deviations: 12.247, 12.472, 29.439, 32.660.

Patient	**Age**	**BP**	**Glucose**	**Cholesterol**
P1	0.000	−0.267	−0.340	0.000
P2	1.225	1.336	1.359	1.225
P3	−1.225	−1.069	−1.019	−1.225

Numerical verification: after normalization every column has mean 0 and standard deviation 1 (shown by the code below). After normalization the contribution to the P1–P2 squared distance becomes Age 17.7%, BP 30.4%, Glucose 34.1% and Cholesterol 17.7%, so no feature dominates. Min–max normalization, $x' = (x - \min)/(\max - \min)$, is an alternative that maps every feature into the range 0 to 1 (for example the Age column becomes 0.5, 1.0 and 0).

Python code :

```
import numpy as np


# Feature matrix X: rows = patients, columns = Age, Blood Pressure, Glucose, Cholesterol
X = np.array([[45, 130, 110, 220],
              [60, 150, 160, 260],
              [30, 120,  90, 180]], dtype=float)


print("Feature matrix X (3 patients x 4 features):")
```

```
print(X)

# ---- (a) effect of raw features: distance between patient 1 and patient 2 ----
diff = X[0] - X[1]
share = diff**2 / np.sum(diff**2) * 100
print("\nRaw difference (P1 - P2):", diff)
print("Contribution of each feature to squared distance (%):", np.round(share, 1))

# ---- (b) z-score normalisation (centering + scaling) ----
mean = X.mean(axis=0)
std = X.std(axis=0)              # population standard deviation
Z = (X - mean) / std
print("\nColumn means:", mean)
print("Column standard deviations:", np.round(std, 3))
print("Z-score matrix:")
print(np.round(Z, 3))

# numerical verification: mean = 0 and std = 1 for every column
print("\nMean of each column after normalisation:", np.round(Z.mean(axis=0), 6))
print("Std of each column after normalisation :", np.round(Z.std(axis=0), 6))

# ---- min-max normalisation (values between 0 and 1) ----
M = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
print("\nMin-Max normalised matrix:")
print(np.round(M, 3))

# distance contribution after normalisation
d_norm = Z[0] - Z[1]
print("\nContribution after z-score (%):",
    np.round(d_norm**2 / np.sum(d_norm**2) * 100, 1))
```

Output:

```
================= RESTART: C:/All Folders/Deep Learning/lab1.py ================
Feature matrix X (3 patients x 4 features):
[[ 45. 130. 110. 220.]
 [ 60. 150. 160. 260.]
 [ 30. 120.  90. 180.]]

Raw difference (P1 - P2): [-15. -20. -50. -40.]
Contribution of each feature to squared distance (%): [ 4.8  8.5 52.9 33.9]

Column means: [ 45.         133.33333333 120.         220.        ]
Column standard deviations: [12.247 12.472 29.439 32.66 ]
Z-score matrix:
[[ 0.    -0.267 -0.34   0.   ]
 [ 1.225  1.336  1.359  1.225]
 [-1.225 -1.069 -1.019 -1.225]]

Mean of each column after normalisation: [ 0. -0. -0.  0.]
Std of each column after normalisation : [1. 1. 1. 1.]

Min-Max normalised matrix:
[[0.5   0.333 0.286 0.5  ]
 [1.    1.    1.    1.   ]
 [0.    0.    0.    0.   ]]

Contribution after z-score (%): [17.7 30.4 34.1 17.7]
```

c) Importance of normalization

- **Distance-based methods** (k-nearest neighbours, K-means, clustering): the Euclidean distance adds squared differences of all features, so a feature with a large range controls the result. After normalization every feature contributes fairly and "similar patients" has a meaningful definition.
- **Gradient-based methods** (linear/logistic regression, neural networks): the gradient of the loss with respect to a weight is proportional to the input value. Features of different scales give gradients of very different sizes, so one learning rate cannot suit all weights. Centered and scaled inputs give a rounder loss surface, allow a larger learning rate, converge faster and avoid saturated activations.

Interpretation: normalization removes the effect of units, so the model learns from the information in the features and not from their numerical size.

TASK 2 – Vector Representation and Matrix Operations | 2 Marks

A hospital assigns different importance weights to age, glucose level, blood pressure and cholesterol when calculating a preliminary risk score.

a) Represent one patient's feature values and the importance weights as vectors and formulate the weighted score using vector notation.

b) Use a small numerical example to calculate the score and interpret it.

Evidence Expected: Vector representation, matrix/vector formulation, numerical multiplication and interpretation.

Answer – Task 2

a) Vector representation and weighted score

Take the feature order (Age, Glucose, Blood pressure, Cholesterol). One patient (normalized values) and the importance weights are column vectors:

$x = [x1, x2, x3, x4]^T = [0.50, 0.80, 0.60, 0.70]^T$

$w = [w1, w2, w3, w4]^T = [0.20, 0.40, 0.30, 0.10]^T$ (weights add up to 1)

The preliminary risk score is the dot product of the two vectors:

$s = w^T x = w1x1 + w2x2 + w3x3 + w4x4$

For many patients the feature matrix X (n × 4) is multiplied by w, which gives all scores at once: $s = Xw$ (an n × 1 vector).

b) Numerical example and interpretation

$s = (0.20 \times 0.50) + (0.40 \times 0.80) + (0.30 \times 0.60) + (0.10 \times 0.70)$

$s = 0.10 + 0.32 + 0.18 + 0.07 = 0.67$

Interpretation: the score 0.67 on a 0 to 1 scale shows a moderately high preliminary risk. Glucose gives the largest part (0.32 out of 0.67, about 48%) because it has the largest weight and a high value. A higher score means higher priority for further examination, but it is only a screening score, not a diagnosis. For three patients with X = [[0.50, 0.80, 0.60, 0.70], [0.90, 0.95, 0.85, 0.90], [0.20, 0.30, 0.25, 0.20]] the matrix product Xw gives scores 0.670, 0.905 and 0.255, so the second patient is ranked first.

Python code:

```
import numpy as np


# Feature order: Age, Glucose, Blood Pressure, Cholesterol (normalised values)
x = np.array([0.50, 0.80, 0.60, 0.70])     # one patient
```

```
w = np.array([0.20, 0.40, 0.30, 0.10])     # importance weights (sum = 1)

# weighted score  s = w^T x
score = np.dot(w, x)
print("Weighted risk score for one patient:", round(score, 4))

# step by step
for name, wi, xi in zip(["Age", "Glucose", "BP", "Cholesterol"], w, x):
    print(name, ":", wi, "x", xi, "=", round(wi * xi, 4))

# matrix form for three patients: s = X w
X = np.array([[0.50, 0.80, 0.60, 0.70],
              [0.90, 0.95, 0.85, 0.90],
              [0.20, 0.30, 0.25, 0.20]])
scores = X @ w
print("\nScores for three patients (X @ w):", np.round(scores, 4))
```

Output:

```
Weighted risk score for one patient: 0.67
Age : 0.2 x 0.5 = 0.1
Glucose : 0.4 x 0.8 = 0.32
BP : 0.3 x 0.6 = 0.18
Cholesterol : 0.1 x 0.7 = 0.07

Scores for three patients (X @ w): [0.67  0.905 0.255]
```

TASK 3 – Probability and Conditional Probability | 6 Marks

In a sample of 1,000 patients, 200 have a particular disease. Of these diseased patients, 160 have elevated glucose. Among the 800 patients without the disease, 120 have elevated glucose.

a) Calculate the probability that a randomly selected patient has elevated glucose. b) Calculate the probability that a patient has the disease given elevated glucose. c) Calculate the probability that a patient has elevated glucose given that the patient has the disease. d) Explain how conditional

probability can support the prioritization of patients for further testing.

Evidence Expected: Event definitions, probability calculations, conditional probability formulae, numerical solutions and healthcare interpretation.

Answer – Task 3

Event definitions: D = patient has the disease, E = patient has elevated glucose. Total N = 1000, P(D) = 200/1000 = 0.20.

	Elevated glucose (E)	Normal glucose	Total
Disease (D)	160	40	200
No disease	120	680	800
Total	280	720	1000

a) Probability of elevated glucose

P(E) = (160 + 120) / 1000 = 280 / 1000 = 0.28

b) Probability of disease given elevated glucose

P(D | E) = P(D ∩ E) / P(E) = (160/1000) / (280/1000) = 160 / 280 = 0.5714

c) Probability of elevated glucose given disease

P(E | D) = P(D ∩ E) / P(D) = (160/1000) / (200/1000) = 160 / 200 = 0.80

Check with Bayes' theorem: P(D | E) = P(E | D) P(D) / P(E) = (0.80 × 0.20) / 0.28 = 0.5714.

Python code :

```
N = 1000              # total patients
D = 200               # patients with disease
ND = 800              # patients without disease
E_and_D = 160         # elevated glucose AND disease
E_and_ND = 120        # elevated glucose AND no disease

# (a) P(E) = (160 + 120) / 1000
P_E = (E_and_D + E_and_ND) / N
print("P(Elevated glucose) =", P_E)
```

```
# (b) P(D | E) = P(D and E) / P(E)
P_D_and_E = E_and_D / N
P_D_given_E = P_D_and_E / P_E
print("P(Disease | Elevated glucose) =", round(P_D_given_E, 4))

# (c) P(E | D) = P(D and E) / P(D)
P_D = D / N
P_E_given_D = P_D_and_E / P_D
print("P(Elevated glucose | Disease) =", round(P_E_given_D, 4))

# check using Bayes theorem: P(D|E) = P(E|D) P(D) / P(E)
print("Check with Bayes theorem      =", round(P_E_given_D * P_D / P_E, 4))
```

Output:

```
================= RESTART: C:/All Folders/Deep Learning/l
P(Elevated glucose) = 0.28
P(Disease | Elevated glucose) = 0.5714
P(Elevated glucose | Disease) = 0.8
Check with Bayes theorem      = 0.5714
>
```

d) Use of conditional probability for prioritization

The overall disease rate is 20%, but among patients with elevated glucose it rises to 57.14%, about 2.9 times higher. Conditional probability P(D | E) therefore tells the hospital how likely a patient is to be diseased after a finding is known, so patients with elevated glucose can be placed higher in the queue for confirmatory tests, while resources are not spent equally on everybody. Different findings can be compared by their conditional probabilities and the patients ranked by the highest P(D | findings). Note that P(E | D) = 0.80 (how many diseased patients show high glucose) is different from P(D | E) = 0.57 (how many high-glucose patients are diseased); confusing the two leads to wrong clinical decisions. Even with elevated glucose 43% of patients are disease-free, so the result is used for prioritization, not for final diagnosis.

TASK 4 – Bayesian Statistics for Disease Classification | 4 Marks

A disease affects 10% of a population. A test is positive for 90% of patients who have the disease and for 5% of patients who do not have it.

a) Using Bayes' theorem, calculate the probability that a patient has the disease given a positive test result.

b) Explain the roles of prior probability and likelihood in the posterior probability.
c) Discuss why Bayesian reasoning is useful when medical information is uncertain.

Evidence Expected: Prior and conditional probabilities, Bayes' theorem, step-by-step calculation and interpretation.

Answer – Task 4

Given: D = disease, + = positive test. Prior P(D) = 0.10, so P(¬D) = 0.90. Likelihoods: P(+ | D) = 0.90 and P(+ | ¬D) = 0.05.

a) Posterior probability using Bayes' theorem

$$P(D \mid +) = P(+ \mid D)\, P(D) / P(+)$$

Step 1 – total probability of a positive test:

$$P(+) = P(+ \mid D)\, P(D) + P(+ \mid \neg D)\, P(\neg D) = (0.90 \times 0.10) + (0.05 \times 0.90) = 0.09 + 0.045 = 0.135$$

Step 2 – posterior:

$$P(D \mid +) = 0.09 / 0.135 = 0.6667 \quad (66.67\%)$$

Interpretation: even with a positive result from a good test, the chance of having the disease is about 67%, not 90%, because the disease is rare (10%). Out of 1000 people, 90 are true positives (0.09 × 1000) and 45 are false positives (0.045 × 1000); 90 out of 135 positives are diseased.

Python code :

```
prior = 0.10                # P(D)
sensitivity = 0.90          # P(+ | D)
false_positive = 0.05       # P(+ | not D)

# total probability of a positive test
p_positive = sensitivity * prior + false_positive * (1 - prior)
print("P(Positive) =", round(p_positive, 4))
```

```
# Bayes theorem
posterior = (sensitivity * prior) / p_positive
print("P(Disease | Positive) =", round(posterior, 4))
print("In percentage =", round(posterior * 100, 2), "%")
```

Output:

```
>
================== RESTART: C:/All Folders/Deep Learning/lab1.py =============
P(Positive) = 0.135
P(Disease | Positive) = 0.6667
In percentage = 66.67 %
>
```

b) Roles of prior probability and likelihood

Posterior $\propto$ Likelihood $\times$ Prior. The **prior** P(D) is what is believed about the disease before seeing the test (here the prevalence, 10%). The **likelihood** P(+ | D) tells how strongly the test result supports the disease hypothesis compared with P(+ | ¬D). The posterior combines both: a high likelihood raises the probability, but a low prior pulls it down. If the prior were 1%, the same test would give a posterior of only about 15%; if the prior were 50%, it would be about 95%.

c) Usefulness of Bayesian reasoning under uncertainty

- It combines prior medical knowledge (prevalence, earlier results) with new evidence in a mathematically correct way.
- Beliefs can be updated repeatedly as new tests or symptoms arrive (the posterior becomes the next prior).
- It outputs a probability, which expresses the degree of uncertainty instead of a simple yes/no decision.
- It avoids the base-rate mistake of assuming that a positive test from a 90% sensitive test means 90% chance of disease, and it works even with small or noisy data.

TASK 5 – Entropy and Information Gain | 3 Marks

A medical dataset contains 100 patient records: 40 are classified as high-risk and 60 as low-risk. a) Calculate the entropy of the risk classification.

b) Explain what the entropy indicates about uncertainty.

c) Explain how information gain can help select a useful feature, such as glucose level.

Evidence Expected: Entropy formula, numerical calculation, interpretation of uncertainty and explanation of information gain.

Answer – Task 5

a) Entropy of the risk classification

Entropy formula (in bits): $H(S) = -\Sigma\, p_i \log_2 p_i$. Here p(high-risk) = 40/100 = 0.4 and p(low-risk) = 60/100 = 0.6.

$$H(S) = -[0.4 \log_2(0.4) + 0.6 \log_2(0.6)]$$

$$H(S) = -[0.4 \times (-1.3219) + 0.6 \times (-0.7370)] = 0.5288 + 0.4422 = 0.9710 \text{ bits}$$

Python code:

```
import math

def entropy(probs):
    """Entropy in bits: H = - sum p log2 p"""
    return -sum(p * math.log2(p) for p in probs if p > 0)

# (a) 40 high-risk and 60 low-risk records
H_parent = entropy([40/100, 60/100])
print("Entropy of risk classification =", round(H_parent, 4), "bits")

# (c) illustrative information gain for a glucose split (assumed split)
# High glucose group  : 50 records -> 35 high-risk, 15 low-risk
# Normal glucose group: 50 records -> 5 high-risk, 45 low-risk
H_high = entropy([35/50, 15/50])
H_normal = entropy([5/50, 45/50])
H_children = (50/100) * H_high + (50/100) * H_normal
IG = H_parent - H_children

print("Entropy (high glucose group)   =", round(H_high, 4))
print("Entropy (normal glucose group) =", round(H_normal, 4))
print("Weighted entropy after split   =", round(H_children, 4))
print("Information gain (glucose)     =", round(IG, 4))
```

Output:

```
================= RESTART: C:/All Folders/Deep Learning/lab1.py ================
Entropy of risk classification = 0.971 bits
Entropy (high glucose group)   = 0.8813
Entropy (normal glucose group) = 0.469
Weighted entropy after split   = 0.6751
Information gain (glucose)     = 0.2958
```

b) Meaning of the entropy value

Entropy measures the uncertainty (impurity) of the class labels. Its value ranges from 0 bits (all records belong to one class, no uncertainty) to 1 bit for two classes (50% / 50%, maximum uncertainty). The value 0.971 is very close to 1, so the dataset is highly mixed and a randomly chosen patient's risk class is difficult to predict without further information.

c) Information gain and feature selection

Information gain is the reduction in entropy obtained after splitting the data by a feature:

$$IG(S, A) = H(S) - \Sigma\ (|Sv| / |S|)\ H(Sv)$$

A feature with high information gain separates high-risk and low-risk patients well, so it is chosen first (for example at the root of a decision tree). Illustration with assumed numbers: if glucose splits the 100 records into a high-glucose group of 50 (35 high-risk, 15 low-risk, entropy 0.8813) and a normal-glucose group of 50 (5 high-risk, 45 low-risk, entropy 0.4690), the weighted entropy after the split is $0.5 \times 0.8813 + 0.5 \times 0.4690 = 0.6751$, and IG = $0.9710 - 0.6751 = 0.2958$ bits. A feature with a larger gain than other features (age, blood pressure, cholesterol) would be selected as the most useful. These group counts are assumed only to demonstrate the method.

TASK 6 – Numerical Computation and Distance Measurement | 3 Marks

After normalization, Patient A is represented by (0.70, 0.80, 0.60) and Patient B by (0.50, 0.75, 0.90), using glucose, blood pressure and cholesterol features.

a) Calculate the Euclidean distance between the two patients.

b) Explain how the distance can indicate similarity between patient records.

c) Explain why normalization is necessary before comparing features with different scales.

Evidence Expected: Distance formula, step-by-step calculation, similarity interpretation and explanation of normalization.

Answer – Task 6

a) Euclidean distance

Formula: $d(A, B) = \sqrt{[\ \Sigma\ (ai - bi)^2\ ]}$. Patient A = (0.70, 0.80, 0.60) and Patient B = (0.50, 0.75, 0.90).

$$d = \sqrt{[\ (0.70 - 0.50)^2 + (0.80 - 0.75)^2 + (0.60 - 0.90)^2\ ]}$$

$$d = \sqrt{[\ 0.04 + 0.0025 + 0.09\ ]} = \sqrt{0.1325} = 0.3640$$

Python code :

```
import math

A = (0.70, 0.80, 0.60)      # (glucose, blood pressure, cholesterol)
B = (0.50, 0.75, 0.90)

# Euclidean distance d = sqrt( sum (a_i - b_i)^2 )
squares = [(a - b) ** 2 for a, b in zip(A, B)]
for name, s in zip(["Glucose", "Blood pressure", "Cholesterol"], squares):
    print(name, "squared difference =", round(s, 4))

distance = math.sqrt(sum(squares))
print("Sum of squared differences =", round(sum(squares), 4))
print("Euclidean distance =", round(distance, 4))
```

Output:

```
================= RESTART: C:/All Folders/Deep Learning/lab1.py ============
Glucose squared difference = 0.04
Blood pressure squared difference = 0.0025
Cholesterol squared difference = 0.09
Sum of squared differences = 0.1325
Euclidean distance = 0.364
```

b) Distance as a measure of similarity

A small distance means that the two patient records have similar feature values, and a large distance means they are different. The maximum possible distance between two points in the unit cube is $\sqrt{3}$ = 1.732, so 0.364 (about 21% of the maximum) shows that A and B are moderately similar. Most of the difference comes from cholesterol (0.09 of the 0.1325 total). The distance can be used to find the nearest patients (k-nearest neighbours), to compare a new patient with past cases, or to group patients (clustering).

c) Need for normalization before comparison

Without normalization the raw features have different units and ranges (for example glucose around 90–200 mg/dL and cholesterol around 150–300 mg/dL against values below 1 for some other measures), so the feature with the largest range would dominate the distance and the result would depend on units. Normalization puts all features on a comparable scale (here 0 to 1), so each feature contributes in proportion to its real difference and the distance becomes a fair similarity measure.

TASK 7 – Bias–Variance Trade-off | 4 Marks

Three disease-risk models are evaluated. Model A has high training and validation error. Model B has very low training error but high validation error. Model C has low and similar training and validation errors.

a) Identify which model has high bias and which has high variance.

b) Explain how model complexity affects bias and variance.

c) Suggest a suitable improvement for Models A and B.

Evidence Expected: Correct identification, bias–variance explanation, comparison of model behaviour and suitable strategies.

Answer – Task 7

a) Identification of bias and variance

Model	Training error	Validation error	Diagnosis
A	High	High	High bias (underfitting)

B	Very low	High (large gap)	High variance (overfitting)
C	Low	Low and similar	Good balance (good generalization)

Model A is too simple, so it cannot fit even the training data (high bias). Model B memorizes the training data including noise, so it fails on new patients (high variance). Model C has low bias and low variance.

b) Effect of model complexity

Expected error = Bias² + Variance + Irreducible noise. When complexity is low, the model makes strong assumptions, so bias is high and variance is low (the model changes little for different training samples). When complexity increases, the model becomes flexible, so bias decreases but variance increases because the model becomes sensitive to the particular training sample and noise. Total error is therefore U-shaped, and the best model lies at the complexity where bias and variance are balanced (Model C).

c) Suitable improvements

Model	Problem	Improvements
A	High bias	Increase model complexity (more layers/neurons, polynomial or interaction features); add more informative features; reduce regularization; train for more epochs.
B	High variance	Collect more training data; apply regularization (L1/L2, dropout); reduce model complexity; early stopping; feature selection; cross-validation; data augmentation.

TASK 8 – Overfitting, Underfitting and Model Capacity | 4 Marks

A simple model fails to capture disease patterns, a moderately complex model performs consistently, and a highly complex model performs almost perfectly on training data but poorly on unseen patients.

a) Identify underfitting, appropriate generalization and overfitting.

b) Explain how model capacity affects the ability to learn complex relationships. c) Explain

why increasing capacity may first improve performance but later cause overfitting.

Evidence Expected: Correct classification of model behaviour, explanation of capacity and relationship with generalization.

Answer – Task 8

a) Identification of model behaviour

Model	Behaviour	Reason
Simple model	Underfitting	Fails to capture the disease patterns; error is high on both training and unseen data.
Moderately complex model	Appropriate generalization	Performs consistently on training and unseen patients.
Highly complex model	Overfitting	Almost perfect on training data but poor on unseen patients; it has memorized noise.

b) Effect of model capacity

Model capacity is the range of functions that a model can represent (it grows with the number of parameters, layers, neurons or polynomial degree). A low-capacity model (for example a straight line) can represent only simple relationships, so it cannot learn complex nonlinear relations between age, glucose, blood pressure and cholesterol. A higher-capacity model can represent more complicated relationships and therefore learn them, provided that enough data are available.

c) Why higher capacity first helps and later hurts

At low capacity the main problem is underfitting (high bias), so adding capacity lets the model learn real patterns and both training and validation error fall. After the point where the real patterns have been learned, the extra capacity is used to fit random noise and special cases of the training sample. Training error keeps decreasing, but validation error starts to rise because the model no longer generalizes, which gives a U-shaped validation curve. The generalization gap (validation error − training error) grows with capacity. The best capacity is therefore chosen using validation data, and regularization, early stopping or more data are used to control overfitting.

TASK 9 – Supervised Learning and Model Evaluation | 3 Marks

A hospital has historical patient records labelled as diseased or disease-free and wants to predict outcomes for new patients.

a) Explain why this is a supervised learning problem.

b) Describe the roles of training and testing sets.
c) If 92 of 100 test patients are classified correctly, calculate accuracy and interpret it.

Evidence Expected: Identification of supervised learning, training/testing explanation, accuracy calculation and interpretation.

Answer – Task 9

a) Why this is supervised learning

The historical records contain input features (age, blood pressure, glucose, cholesterol, etc.) together with known output labels (diseased or disease-free). The model learns a mapping f: X → y from labelled examples and then predicts the label of new patients. Since the output is a category, it is a supervised classification problem.

b) Roles of training and testing sets

- **Training set** (for example 70–80% of the records): used to fit the model parameters by minimizing the error between predictions and known labels.
- **Testing set** (the remaining 20–30%): kept unseen during training and used only once to estimate how well the model will perform on new patients, so overfitting can be detected. A validation set (or cross-validation) is also used to tune hyperparameters.

c) Accuracy

Accuracy = Correct predictions / Total predictions = 92 / 100 = 0.92 = 92%

Interpretation: the model classifies 92 of every 100 new patients correctly, and the error rate is 8%. This is good, but accuracy alone can mislead when the classes are imbalanced (if only 8% of patients were diseased, a model that always predicts "disease-free" would also score 92%). In healthcare, missed disease cases (false negatives) are costly, so the confusion matrix, precision, recall (sensitivity) and F1-score should also be checked.

Python code :

```
correct = 92
total = 100


accuracy = correct / total
print("Accuracy =", accuracy)
print("Accuracy in percentage =", accuracy * 100, "%")
print("Error rate =", round(1 - accuracy, 2))
```

Output:

```
================= RESTART: C:/All Folders/Deep Learning/lab1.py ==
Accuracy = 0.92
Accuracy in percentage = 92.0 %
Error rate = 0.08
>
```

TASK 10 – Unsupervised Learning and Patient Grouping | 4 Marks

A hospital has patient records containing age, blood pressure, glucose and cholesterol but no predefined risk labels.

a) Explain why unsupervised learning is suitable.

b) Describe how K-means can group patients with similar characteristics.

c) Explain how distance can be used to measure similarity.

d) Describe how the clusters could support further clinical investigation without replacing medical diagnosis.

Evidence Expected: Learning approach, clustering procedure, distance-based similarity and meaningful interpretation.

Answer – Task 10

a) Why unsupervised learning is suitable

The records have no predefined risk labels, so there is no target value to learn from. Unsupervised learning finds hidden structure (groups of similar patients) in the features alone, without labels.

b) K-means grouping

K-means divides the patients into K clusters by minimizing the within-cluster sum of squares $J = \Sigma k \; \Sigma x \in Ck \; ||x - \mu k||^2$. Steps:

- Normalize the features and choose the number of clusters K (for example using the elbow method).
- Choose K initial centroids (random patients or k-means++).
- Assignment step: assign every patient to the nearest centroid (Euclidean distance).
- Update step: recompute each centroid as the mean of the patients assigned to it.
- Repeat the assignment and update steps until the centroids no longer change (convergence).

Small example with six normalized patients (Age, BP, Glucose, Cholesterol) and K = 2, starting with P1 and P2 as centroids:

Patient	Age	BP	Glucose	Cholesterol
P1	0.20	0.25	0.30	0.20
P2	0.25	0.30	0.25	0.30
P3	0.30	0.20	0.35	0.25
P4	0.80	0.85	0.90	0.80
P5	0.85	0.75	0.80	0.90
P6	0.90	0.80	0.85	0.85

Iteration 1 gives clusters {P1, P3} and {P2, P4, P5, P6}; iteration 2 moves P2 to the first cluster, giving {P1, P2, P3} with centroid (0.25, 0.25, 0.30, 0.25) and {P4, P5, P6} with centroid (0.85, 0.80, 0.85, 0.85). Iteration 3 changes nothing, so the algorithm has converged.

Python code :

```
import numpy as np

# Normalised patients: Age, Blood Pressure, Glucose, Cholesterol
X = np.array([[0.20, 0.25, 0.30, 0.20],    # P1
              [0.25, 0.30, 0.25, 0.30],    # P2
              [0.30, 0.20, 0.35, 0.25],    # P3
              [0.80, 0.85, 0.90, 0.80],    # P4
              [0.85, 0.75, 0.80, 0.90],    # P5
              [0.90, 0.80, 0.85, 0.85]])   # P6

K = 2
centroids = X[[0, 1]].copy()              # start with P1 and P2 as centroids

for it in range(1, 10):
    # assignment step: nearest centroid (Euclidean distance)
    dist = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
    labels = dist.argmin(axis=1)

    # update step: mean of each cluster
    new_centroids = np.array([X[labels == k].mean(axis=0) for k in range(K)])

    print("Iteration", it, "-> cluster of P1..P6:", labels + 1)
    print("Centroids:\n", np.round(new_centroids, 3))

    if np.allclose(new_centroids, centroids):
        print("Centroids did not change: K-means has converged.")
        break
    centroids = new_centroids

print("\nFinal clusters:")
for k in range(K):
    members = ["P" + str(i + 1) for i in np.where(labels == k)[0]]
```

```
print("Cluster", k + 1, ":", members)
```

Output:

```
================= RESTART: C:/All Folders/Deep Learning/lab1.py ================
Iteration 1 -> cluster of P1..P6: [1 2 1 2 2 2]
Centroids:
 [[0.25  0.225 0.325 0.225]
 [0.7   0.675 0.7   0.712]]
Iteration 2 -> cluster of P1..P6: [1 1 1 2 2 2]
Centroids:
 [[0.25 0.25 0.3  0.25]
 [0.85 0.8  0.85 0.85]]
Iteration 3 -> cluster of P1..P6: [1 1 1 2 2 2]
Centroids:
 [[0.25 0.25 0.3  0.25]
 [0.85 0.8  0.85 0.85]]
Centroids did not change: K-means has converged.

Final clusters:
Cluster 1 : ['P1', 'P2', 'P3']
Cluster 2 : ['P4', 'P5', 'P6']
>
```

c) Distance as similarity

The Euclidean distance $d(x, y) = \sqrt{\Sigma(xi - yi)^2}$ between two patient vectors measures how different they are: patients inside the same cluster are close to each other and close to their centroid, and patients in different clusters are far apart. In the example, P1 and P2 have a distance of about 0.13, whereas P1 and P4 have a distance of about 1.2, so they fall into different clusters. Features must be normalized so that the distance is not dominated by one feature.

d) Support for clinical investigation without replacing diagnosis

The clusters show groups of patients with similar profiles (for example a group with consistently high glucose, blood pressure and cholesterol). Doctors can examine such groups to decide who needs earlier screening or follow-up, design targeted preventive programmes, and form new medical hypotheses about risk patterns. The clusters have no disease label, so they are not diagnoses: every group must be reviewed and interpreted by clinicians, validated with clinical tests (and with cluster quality measures such as the silhouette score), and the final diagnosis is always made by a medical professional.

TASK 11 – Comparison of Supervised and Unsupervised Learning | 4 Marks

Dataset A contains patient features with disease labels, while Dataset B contains patient features without labels.

a) Identify the suitable learning paradigm for each dataset and justify your

choice. b) Compare their objectives and outputs.

c) Recommend an approach for predicting disease status in a new patient and justify the recommendation.

Evidence Expected: Correct identification, comparison of objectives and outputs, and context-based justification.

Answer – Task 11

a) Suitable learning paradigm for each dataset

- **Dataset A (features with disease labels): supervised learning**, because every record has a known target, so a classifier can learn the relationship between features and disease status.
- **Dataset B (features without labels): unsupervised learning**, because there is no target; clustering or dimensionality reduction can discover groups and structure in the features.

b) Comparison of objectives and outputs

Aspect	**Supervised (Dataset A)**	**Unsupervised (Dataset B)**
Input data	Features + labels	Features only
Objective	Learn mapping X → y to predict labels for new patients	Discover hidden structure, groups or patterns in X
Output	Predicted class or probability (diseased / disease-free)	Cluster membership, centroids, low-dimensional representation
Evaluation	Accuracy, precision, recall, F1 on a test set	Silhouette score, within-cluster variance, clinical interpretation
Typical methods	Logistic regression, decision tree, neural network	K-means, hierarchical clustering, PCA

c) Recommended approach for predicting disease status of a new patient

A supervised classification model trained on Dataset A is recommended (with normalized features and a train/test split to measure generalization), because predicting disease status requires labelled examples; clusters from Dataset B have no disease meaning on their own. The unlabeled Dataset B can

still help: its clusters or distances can be added as extra features, or used in a semi-supervised way, to improve the model, and the clusters can be used afterwards to explain patient groups. The final prediction is a probability that supports, but does not replace, the doctor's decision.

TASK 12 – Design Challenge: Selecting and Adapting a Learning Strategy | 8 Marks

Changed Requirement

The hospital initially analyzes 10,000 patient records. It must now process 1,000,000 records, predict disease risk, discover patient groups, and handle missing values and differences in feature scales. Decide whether to retain, modify or redesign the analytical strategy.

A. Impact Analysis | 2 Marks

a) Explain how increasing the dataset size affects data representation, numerical computation and learning efficiency.

b) Discuss why normalization, missing-value handling and data variability are important.

B. Evaluation of the Original Approach | 2 Marks

a) Evaluate whether a single supervised approach is sufficient for both disease prediction and patient group discovery.

b) Explain the limitations of labelled learning when useful patterns also exist in unlabelled records.

C. Adaptation or Redesign | 2 Marks

a) Propose a strategy combining supervised classification and unsupervised clustering. b) Explain how probability, distance measures, normalization and model capacity can be incorporated.

D. Evidence-Based Defence | 2 Marks

a) Propose suitable experiments and evaluation measures to compare the redesigned approach with the original.

b) Justify the final design using mathematical reasoning, generalization, computational needs and data characteristics.

Evidence Expected: Analysis of data scale and quality, justified learning strategy, mathematical reasoning and suitable evaluation plan.

Answer – Task 12: Design Challenge

Decision: the original strategy should be **modified** into a scalable hybrid design (preprocessing + clustering on all records + supervised classification), not simply retained and not rebuilt from nothing. The supervised classifier is kept, but data handling is redesigned and an unsupervised component is added.

A. Impact Analysis

a) Effect of increasing the dataset from 10,000 to 1,000,000 records

- **Data representation:** the feature matrix grows from 10,000 × d to 1,000,000 × d. With d = 4 and 8 bytes per value it needs about 32 MB, but with d = 100 it needs about 800 MB, so efficient storage (float32, sparse formats, chunked loading) is needed.
- **Numerical computation:** one pass over the data costs O(n·d), which is 100 times more; matrix operations must be vectorized (NumPy/GPU), and sums of millions of floating-point numbers can lose precision, so stable algorithms and sufficient precision are required.
- **Learning efficiency:** full-batch training and standard K-means (cost O(n·K·d·iterations)) become slow, so mini-batch gradient descent and mini-batch K-means are used. The benefit of more data is lower variance and less overfitting, which allows a higher-capacity model.

b) Importance of normalization, missing-value handling and data variability

- **Normalization:** needed so that distances and gradients are not dominated by large-scale features; mean and standard deviation are computed on the training data only and then applied to validation and test data (they can also be computed in a streaming way for 1,000,000 records).
- **Missing values:** deleting records wastes data and may introduce bias; imputation (mean/median, nearest neighbours, model-based) plus a missing-indicator feature keeps the records. Wrong handling distorts the means, distances and probabilities.
- **Data variability:** differences between hospitals, outliers, noise and different patient groups make the data heterogeneous; robust scaling, outlier checks and stratified sampling help the model to generalize.

B. Evaluation of the Original Approach

a) Is a single supervised approach sufficient?

No. A supervised model can only predict the labelled outcome (disease risk). It cannot discover patient groups that were not defined beforehand, so patient group discovery needs an unsupervised method such as clustering. One approach alone cannot satisfy both objectives.

b) Limitations of labelled learning

- Labels are expensive and slow to obtain (they need clinical confirmation), so only a small part of the 1,000,000 records is usually labelled.
- The labelled subset may be biased and not representative, so the model may generalize poorly.
- The structure, correlations and sub-groups that exist in the unlabelled records are ignored, although they contain useful information.
- Label noise and a small labelled sample increase variance and the risk of overfitting.

C. Adaptation or Redesign

a) Proposed combined strategy

Step	Stage	Method
1	Data cleaning	Handle missing values (imputation + missing-indicator), remove invalid records and outliers
2	Normalization	Z-score or robust scaling fitted on training data and applied to all records
3	Unsupervised stage	Mini-batch K-means (optionally after PCA) on all 1,000,000 records to discover patient groups
4	Supervised stage	Train a classifier (logistic regression or neural network) on the labelled records using the normalized features and the cluster information (cluster id / distance to centroids)
5	Output	Calibrated disease-risk probability plus patient group for each patient, reviewed by clinicians

b) Role of the mathematical concepts

- **Probability:** the classifier gives P(disease | features), which can be combined with the prior prevalence using Bayes' theorem and checked for calibration.
- **Distance measures:** Euclidean distance on normalized features is used by K-means to group similar patients and to compute the distance of a patient to each cluster centroid.

- **Normalization:** puts all features on the same scale so that distances are meaningful and gradient descent converges quickly.
- **Model capacity:** selected using validation data and controlled with regularization and early stopping; with 1,000,000 records a higher-capacity model can be used without much overfitting.

D. Evidence-Based Defence

a) Experiments and evaluation measures

Experiment	Comparison	Measures
1. Main comparison	Original (single supervised model) against redesigned hybrid model on the same held-out test set	Accuracy, precision, recall, F1-score, ROC-AUC, calibration (Brier score)
2. Ablation study	Remove one component at a time: normalization, imputation, cluster features	Change in F1 and AUC
3. Cluster quality	Compare K values and repeated runs	Silhouette score, within-cluster sum of squares, stability, clinical interpretability
4. Generalization	Stratified k-fold cross-validation; compare training and validation error	Mean and standard deviation of scores, generalization gap
5. Scalability	Run on 10,000, 100,000 and 1,000,000 records	Training time, memory use, throughput

Statistical tests (for example a paired t-test or McNemar's test) are used to confirm that any improvement is not due to chance.

b) Justification of the final design

- **Mathematical reasoning:** normalization makes Euclidean distance and gradient descent well-behaved; Bayes' theorem and calibrated probabilities allow risk to be interpreted correctly; K-means minimizes within-cluster variance on all records.

- **Generalization:** more data lowers the variance of the estimates, regularization and validation control the capacity, and unlabelled data add information about the structure of the population, so the model is less likely to overfit the small labelled set.
- **Computational needs:** mini-batch methods have a cost that grows linearly with the number of records, so 1,000,000 records can be processed with reasonable memory and time.
- **Data characteristics:** missing values, different scales and heterogeneous patient groups are handled explicitly by the preprocessing and clustering stages.

Therefore the modified hybrid strategy is better justified than the original single supervised approach.